Introduction to change point detection

Computer lab worksheet

Euan McGonigle

Instructions

- Full solutions are given below. If you feel you're spending too much time on any question (particularly Question 1), you can read the solution and move on to questions 2 – 4.
- Likewise, feel free to "vibe code" if you want to save time, or use existing R package functionality rather than writing your own code.
- Bonus questions: Q 1(e) is a bonus, as is Question 5. Feel free to skip these if you want.
- For any functions you're not sure how to use, use `?function_name` in R to get help (or ask me!).
- In the time we have, I don't expect you to finish all questions. If you can, try to finish at least Q2.

Question 1: Creating a test statistic to estimate change points

(a) Without using pre-existing change point detection libraries, write a function to calculate **either**:

- the CUSUM statistic vector $\{T_{0,k,n}\}_{k=1}^{n-1}$ for a change point in the mean,
- the MOSUM statistic vector $\{T_G(k)\}_{k=G}^{n-G}$ for a change point in the mean, for a given bandwidth G.

Solution:

```r
CUSUM.calc <- function(x){

  n <- length(x)
  I.plus <- I.minus <- I.prod <- rep(0, n - 1)
  I.plus[1] <- sqrt(1 - 1/n) * x[1]
  I.minus[1] <- 1/sqrt(n^2 - n) * sum(x[2:n])
  for (k in 1:(n - 2)) {
      factor <- sqrt((n - k - 1) * k/(k + 1)/(n - k))
      I.plus[k + 1] <- I.plus[k] * factor + x[k + 1] * sqrt(1/(k + 1) - 1/n)
      I.minus[k + 1] <- I.minus[k]/factor - x[k + 1]/sqrt(n^2/(k + 1) - n)
  }
  x.CUSUM <- I.plus - I.minus
  return(abs(x.CUSUM))

}
```

```r
MOSUM.calc <- function(x, G){

  n <- length(x)

  sums <- rep(NA, n)
  currentSum <- sum(x[1:G])

  sums[1] <- currentSum
  for (k in 2:(n-G)) {
    currentSum <- currentSum + x[k + G - 1]
    currentSum <- currentSum - x[k - 1]
    sums[k] <- currentSum
  }

  x.MOSUM  <- c(rep(NA, G - 1), sums[(G + 1):n] - sums[1:(n - G)], NA)/sqrt(2*G)

  return(abs(x.MOSUM))

}
```

(b) To set a threshold for declaring changes, we need to know the noise level σ. In reality this is unknown, so we must estimate it from the data – but the change points themselves will inflate a naive estimate. Consider the following candidate estimators of σ:

- **(i)** the sample standard deviation of the whole series, `sd(x)`;
- **(ii)** the sample standard deviation of only the first 20 observations, `sd(x[1:20])`;

- **(iii)** the median absolute deviation of the data, `mad(x)`;
- **(iv)** the median absolute deviation of the (scaled) first differences, `mad(diff(x) / sqrt(2))`.

For each option, think about whether it would be biased by the presence of change points or a trend. Which are good choices, and which are bad? Pick one estimator to take forward and use in the rest of the lab.

Hint: differencing the series removes a piecewise-constant mean, leaving the noise plus a spike at each change point; the median absolute deviation is barely affected by a small number of large spikes. What is the factor of $\sqrt{2}$ doing?

Solution: The key is to find an estimator that is not thrown off by the change points (or by a trend):

(i) `sd(x)` is a **bad** choice. The changes in mean (and any trend) are counted as part of the spread, so the estimate is positively biased – often badly so.

(iii) `mad(x)` is more robust to the occasional outlier than `sd(x)`, but it is **still biased** here: a change in mean moves a large fraction of the observations away from the overall median, and the MAD only tolerates contamination affecting less than half of the data. Robustness to outliers is not the same as robustness to change points.

(ii) `sd(x[1:20])` is **defensible but fragile**. It is unbiased *if* the first 20 observations form a single clean segment with no change point and no trend, but an early change or a trend ruins it, and it throws away almost all of the data.

(iv) `mad(diff(x) / sqrt(2))` is the **best** of the four, and is the one we use. First-differencing removes a piecewise-constant mean, so the differenced series is just noise except for a spike at each change point. As there are only a few change points, these behave as rare outliers that the MAD ignores. The factor $\sqrt{2}$ corrects for $\mathrm{Var}(x_{t+1} - x_t) = 2\sigma^2$, so that the estimator targets σ rather than $\sqrt{2}\,\sigma$.

More sophisticated *local* estimators can also be used – for example the local variance estimators in the `mosum` package, $\hat{\sigma}^2(k) = (\hat{\sigma}^2_{(k-G+1):k} + \hat{\sigma}^2_{(k+1):(k+G)})/2$, where $\hat{\sigma}^2_{s:e}$ is the sample variance of the data from s to e.

(c) Implement your chosen method of estimating σ, and add the functionality to your CUSUM or MOSUM function from part (a) so that the test statistic calculation is scaled by your estimate $\hat{\sigma}$.

Solution:

```r
sigma.est <- function(x){

  x.diff <- diff(x)/sqrt(2) #scaled so that var(x) = var(transformed x)

  sigma.hat <- mad(x.diff)

  return(sigma.hat)

}

#answer for the MOSUM method:

MOSUM.calc2 <- function(x, G){

  x.MOSUM <- MOSUM.calc(x, G)
  sigma.hat <- sigma.est(x)

  return(x.MOSUM/sigma.hat)
}


CUSUM.calc2 <- function(x){

  x.CUSUM <- CUSUM.calc(x)
  sigma.hat <- sigma.est(x)

  return(x.CUSUM/sigma.hat)

}
```

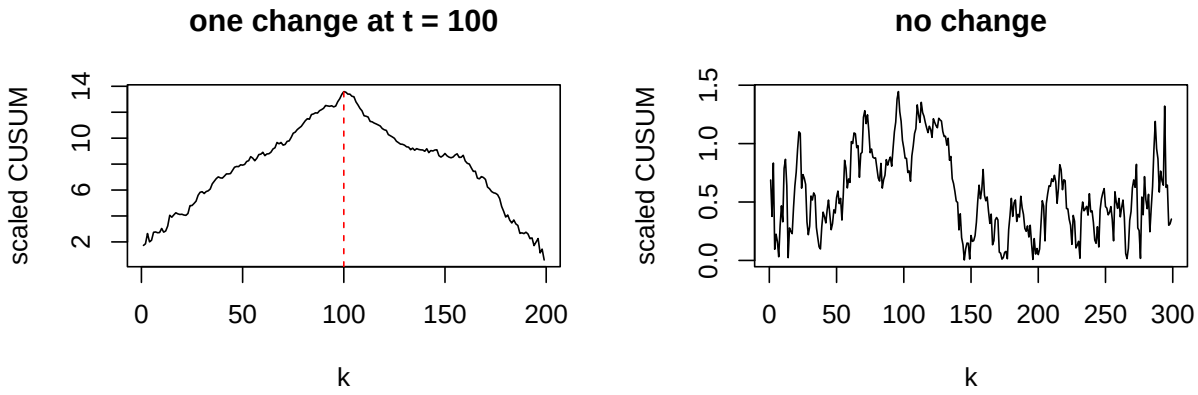
Confirm your function behaves sensibly on data where you already know the answer, with two controlled tests:

One change point. Simulate a series with one known change in mean, `set.seed(1); x <- c(rnorm(100), rnorm(100, mean = 2))`, which changes at $t = 100$. Your (scaled) statistic should peak clearly near $t = 100$.

No change point. Now simulate a series with **no** change at all, `set.seed(1); x <- rnorm(300)`. There is nothing to find, so your statistic should mostly stay close to zero.

Solution: Plotting the scaled CUSUM statistic for each case:

```r
par(mfrow = c(1, 2))

# (1) a series with a known change at t = 100
set.seed(1)
x <- c(rnorm(100), rnorm(100, mean = 2))
plot(CUSUM.calc2(x), type = "l", xlab = "k", ylab = "scaled CUSUM",
     main = "one change at t = 100")
abline(v = 100, col = "red", lty = 2)

# (2) pure noise, no change
set.seed(1)
z <- rnorm(300)
plot(CUSUM.calc2(z), type = "l", xlab = "k", ylab = "scaled CUSUM",
     main = "no change")
```



For the first series the statistic peaks sharply at $t = 100$, confirming the function works; for pure noise it stays small throughout, since there is nothing to detect.

(d) What is the computational complexity of the code you wrote in part (a)? How fast could it be? If you have time, verify your answer empirically: use `system.time()` to time your function on series of increasing length (say $n = 10^2, 10^3, 10^4, 10^5$) and check how the runtime grows.

Solution: By using sequential updates, both methods can be computed on $O(n)$ computational complexity.

We can check this empirically – the elapsed time should grow roughly in proportion to n (multiplying by about 10 each time n does):

```r
for (nn in c(1e2, 1e3, 1e4, 1e5)) {
  set.seed(1)
  z <- rnorm(nn)
  cat("n =", nn, " time =", system.time(CUSUM.calc(z))["elapsed"], "s\n")
}
```

```
n = 100  time = 0 s
n = 1000  time = 0 s
n = 10000  time = 0.002 s
n = 1e+05  time = 0.023 s
```

(e) (Bonus) For simultaneous estimation of multiple change points with the MOSUM approach, we can calculate all $\hat{\theta}$ that satisfy:

$$T_G(\hat{\theta}) > D \quad \text{and} \quad \hat{\theta} = \text{argmax}_{k:\,|k-\hat{\theta}|\leq \eta G} T_G(k).$$

for some window fraction $\eta \in (0, 1)$ and a threshold D. That is, $\hat{\theta}$ is declared a change point if it is a local maximiser of $T_G(k)$ over a sufficiently large interval of radius ηG, at which the threshold D is exceeded.

Extend the MOSUM function to return as output the change point estimators satisfying this criterion.

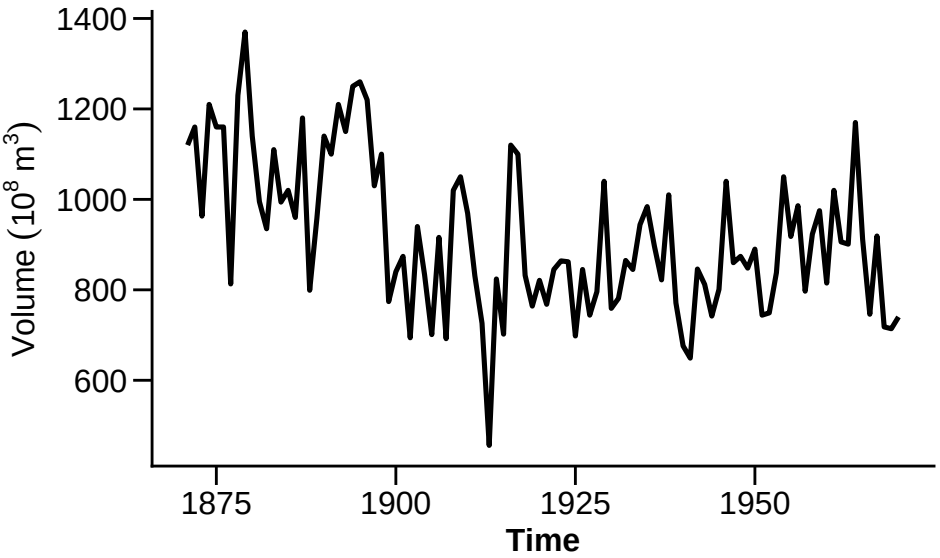
Solution:

```r
MOSUM.calc3 <- function(x, G, eta, D){

  x.MOSUM <- MOSUM.calc2(x, G)

  n <- length(x)
  cpt.ests <- numeric(0)
  window_length <- floor(eta*G)

  exceedings <- (x.MOSUM > D)

  localMaxima <- (c((diff.default(x.MOSUM) < 0), NA) & c(NA, diff.default(x.MOSUM) > 0))
  candidates <- which(exceedings & localMaxima)

  for (j in seq_len(length(candidates))){

    k_star <- candidates[j]
    m_star <- x.MOSUM[k_star]
    left_thresh <- max(G, k_star - window_length)
```

```
    right_thresh <- min(n-G, k_star + window_length)
    largest <- TRUE
    for (l in left_thresh:right_thresh) {
      if (x.MOSUM[l] > m_star) {
        largest <- FALSE
        break
      }
    }
    if (largest) {
      cpt.ests <- c(cpt.ests, k_star)
    }
  }

  return(cpt.ests)
}
```

Question 2: Nile annual river flow

In this question you will use the code you've written so far to analyse data collected on the river Nile. Data in the file `nile_volume.txt` records measurements of the annual volume (in units $10^8 m^3$) of discharge from the Nile River at Aswan for the years 1871 to 1970. The measurements are of meteorological importance as evidence of a possible abrupt change in the rainfall levels around the turn of the 20th century.

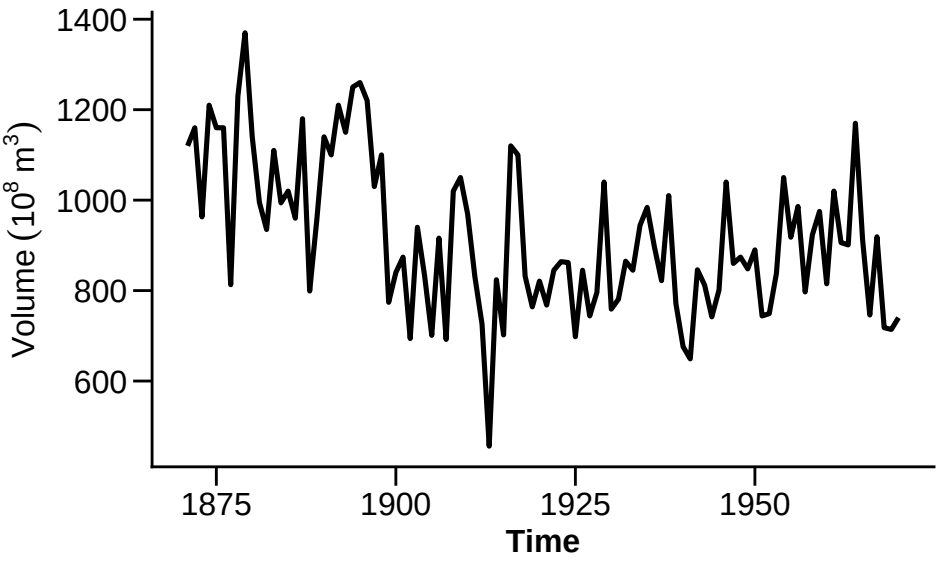


For whichever method (CUSUM or MOSUM) you did not code up in Q1, you can use the code in the answers for this question.

(a) Load the data into R from the `nile_volume.txt` file, and plot it. By eye, where does it look like there could be a change in mean?

Solution:

```
library(ggplot2)

nile.data <- read.table("nile_volume.txt",
                        header = TRUE)

p1 <- ggplot(data = nile.data, aes(x=year,y=volume))+
  geom_line(data=nile.data,color="black",linewidth=0.9)+
  theme_classic()+
  labs(x="Time",y=expression(Volume~(10^8~m^3))) +
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title = element_text(size=18,face="bold"))

p1
```



Around 1900 it looks like there could be a drop in the volume of river flow.

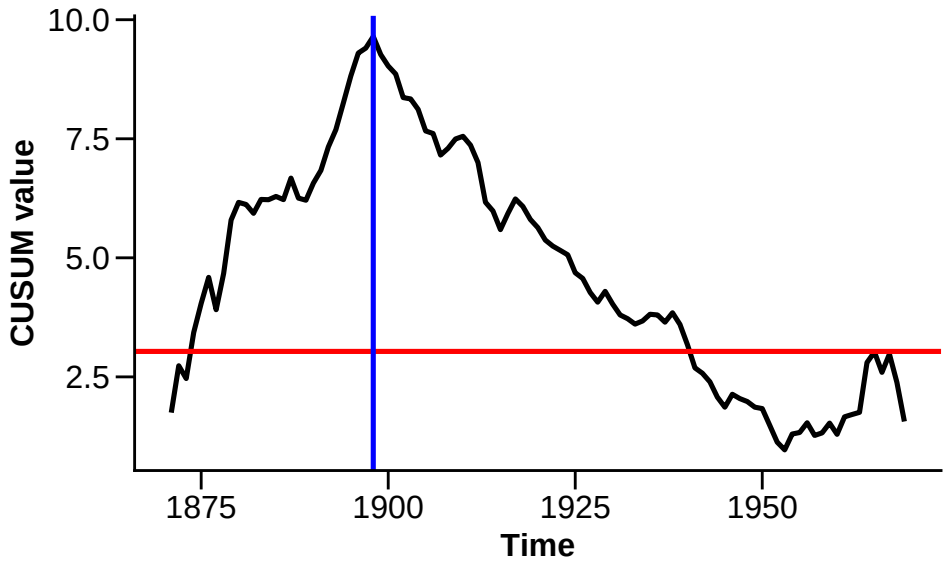
(b) Calculate the CUSUM statistic for the Nile volume data. Using the threshold $D = \sqrt{2\log(n)}$, perform a test to decide if there is a change point. If there is one, where is it?

Solution:

```
nile.CUSUM <- CUSUM.calc2(nile.data$volume)

max(nile.CUSUM) > sqrt(2 * log(nrow(nile.data))) # check if above threshold
```

```
[1] TRUE
```

```
nile.cpt <- which.max(nile.CUSUM)

CUSUM.df <- data.frame(year = nile.data$year[1:99], CUSUM = nile.CUSUM)

CUSUM.plot <- ggplot(data = CUSUM.df, aes(x=year,y=CUSUM))+
  geom_line(data=CUSUM.df,color="black",linewidth=0.9)+
  theme_classic()+
  labs(x="Time",y="CUSUM value") +
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title = element_text(size=18,face="bold"))+
    geom_hline(yintercept = sqrt(2*log(100)), color = "red", linewidth =0.9)+
    geom_vline(xintercept = CUSUM.df$year[nile.cpt], color = "blue", linewidth =0.9)


CUSUM.plot
```



The change point is located at year 1898.

(c) Calculate the MOSUM statistic for the Nile volume data using a bandwidth $G = 25$. Using the threshold $D = 3.4$, compute the change point estimators:

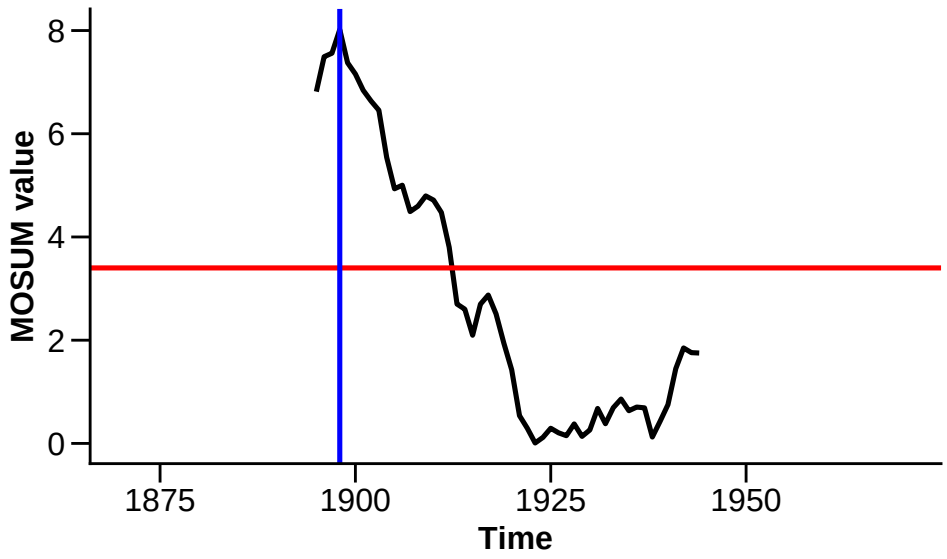
- either by eye by plotting the MOSUM test statistic, or
- using the code from Q1 (e) with $\eta = 0.4$.

Does this agree with your answer from part (b)?

(The value $D = 3.4$ is not arbitrary: it is approximately the critical value that controls the false-alarm probability at the 5% chosen significance level.)

Solution:

```
nile.MOSUM <- MOSUM.calc2(nile.data$volume, G = 25)

nile.cpt2 <- which.max(nile.MOSUM)

MOSUM.df <- data.frame(year = nile.data$year, MOSUM = nile.MOSUM)

MOSUM.plot <- ggplot(data = MOSUM.df, aes(x=year,y=MOSUM))+
  geom_line(data=MOSUM.df,color="black",linewidth=0.9)+
  theme_classic()+
  labs(x="Time",y="MOSUM value") +
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title = element_text(size=18,face="bold"))+
    geom_hline(yintercept = 3.4, color = "red", linewidth =0.9)+
    geom_vline(xintercept = MOSUM.df$year[nile.cpt2], color = "blue", linewidth =0.9)


MOSUM.plot
```

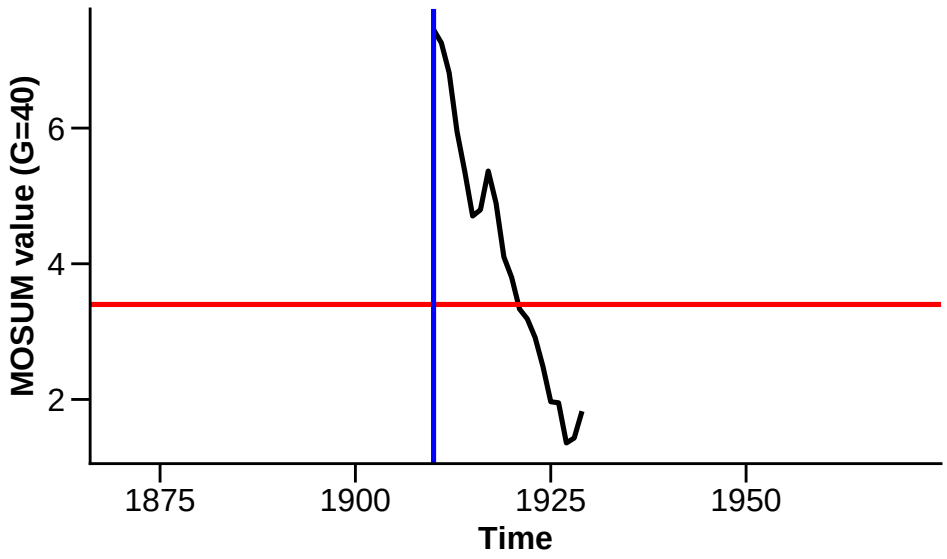


The MOSUM change point is estimated at year 1898, which agrees with the CUSUM estimate.

(d) Now use the MOSUM method with bandwidth $G = 40$. How does your answer differ from part (c)? State the general trade-off in choosing the bandwidth G: what do you gain, and what do you lose, as G increases?

Solution:

```
nile.MOSUM <- MOSUM.calc2(nile.data$volume, G = 40)

nile.cpt <- which.max(nile.MOSUM)

MOSUM.df <- data.frame(year = nile.data$year, MOSUM = nile.MOSUM)

MOSUM.plot <- ggplot(data = MOSUM.df, aes(x=year,y=MOSUM))+
  geom_line(data=MOSUM.df,color="black",linewidth=0.9)+
  theme_classic()+
  labs(x="Time",y="MOSUM value (G=40)") +
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title = element_text(size=18,face="bold"))+
    geom_hline(yintercept = 3.4, color = "red", linewidth =0.9)+
    geom_vline(xintercept = MOSUM.df$year[nile.cpt], color = "blue", linewidth =0.9)


MOSUM.plot
```



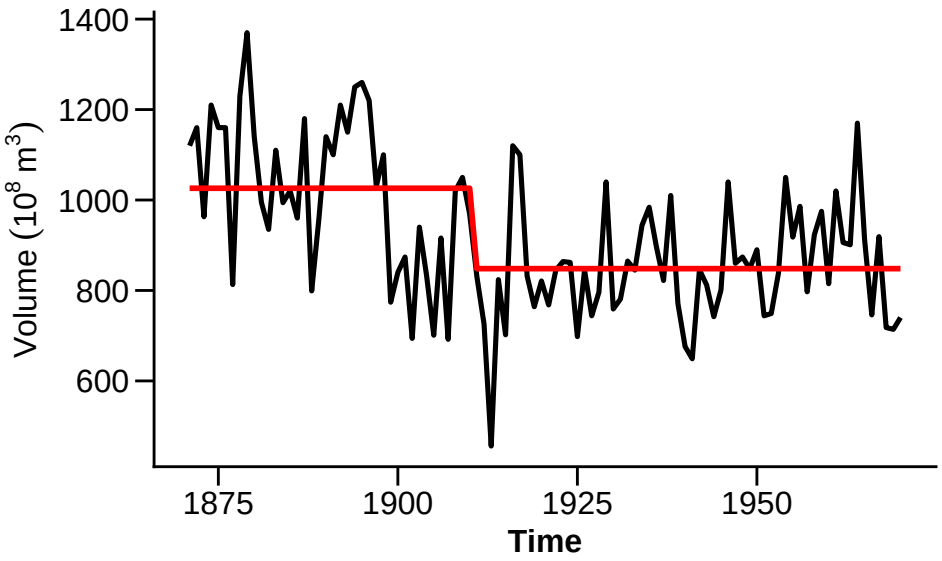
The change point is now estimated as 1910. Without making any modifications to the way the test statistic is calculated, the first time point we can find the change is at year 1910, which is later than the estimate from part (c). This highlights the need to pick a good bandwidth! In general, a **larger** G averages over more observations, so the test is more powerful against changes in long segments and less affected by noise – but it cannot separate changes that are closer together than G, and it cannot locate a change within G of either end of the series. A **smaller** G is more local: it can pick out short segments and changes near the boundaries, but is noisier and more prone to false alarms.

(e) The estimated mean signal $\hat{f}_t$ can be calculated using sample means of the segments defined by the estimated change points. For the CUSUM method, add the estimated $\hat{f}_t$ to your plot from part (a).

Solution

```
calc.mean.func <- function(x, change.loc){

  no.cpts <- length(change.loc)
  n <- length(x)
  change.loc <- c(change.loc,n)
  fitted.mean <- rep(0,n)

  if(no.cpts==0){
    fitted.mean[1:n] <- mean(x)
  }
  else if(no.cpts==1){
```

```
    fitted.mean[1:change.loc[1]] <- mean(x[1:change.loc[1]])
    fitted.mean[(change.loc[1]+1):n] <- mean(x[(change.loc[1]+1):n])
  }
  else if(no.cpts>1){
    fitted.mean[1:change.loc[1]] <- mean(x[1:change.loc[1]])
    for (segs in 1:no.cpts){
      fitted.mean[(change.loc[segs]+1):change.loc[segs+1]] <- mean(x[(change.loc[segs]+1):cha
    }
  }

  return(fitted.mean)
}

nile.mean <- calc.mean.func(nile.data$volume, change.loc = nile.cpt)

mean.df <- data.frame(year = nile.data$year, f_hat = nile.mean)

p1 <- ggplot(data = nile.data, aes(x=year,y=volume))+
  geom_line(data=nile.data,color="black",linewidth=0.9)+
  geom_line(data=mean.df,color="red",linewidth=1, aes(x=year,y=f_hat))+
  theme_classic()+
  labs(x="Time",y=expression(Volume~(10^8~m^3))) +
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title = element_text(size=18,face="bold"))

p1
```



Question 3: Using the mosum and changepoint packages

(a) Load the `changepoint` and `mosum` R packages into your R workspace. Using the help files, get acquainted with the `cpt.mean()` and `mosum()` functions.

```
library(changepoint)
library(mosum)
```

(b) Setting the argument `method = "AMOC"`, use the `cpt.mean()` function on the Nile data set. Do the results agree with your answer from question 2(b)?

Solution:

```
nile.cpts.cusum <- cpt.mean(nile.data$volume, method = "AMOC")
nile.data$year[nile.cpts.cusum@cpts]
```

```
[1] 1898 1970
```

Yes: same answer as before (`cpt.mean()` also returns the last time index).

(c) Use the `cpt.mean()` function to apply the PELT algorithm on the Nile data set. Is the answer the same as part (b)?

Note: you will need to standardise the data first (subtract the mean and divide by the standard deviation). (Why does this matter? By default `cpt.mean()` looks for changes in mean assuming the data has variance 1; if the series has a different variance, then the penalty is scaled incorrectly.)

Solution:

```
#need to standardise the data first!
nile.cpts.pelt <- cpt.mean((nile.data$volume - mean(nile.data$volume))/
                           sd(nile.data$volume), method = "PELT")
nile.data$year[nile.cpts.pelt@cpts]
```

```
[1] 1898 1970
```

Yes: same answer as before.

(d) By investigating the output returned by the `cpt.mean()` function, what was the value of the penalty function used?

Solution:

```
nile.cpts.pelt@pen.value
```

```
[1] 13.81551
```

(e) Using the bandwidth $G = 25$, use the `mosum()` function on the Nile data set. Do the results agree with your answer from question 2(c)?

Solution:

```
nile.cpts.mosum <- mosum(as.numeric(nile.data$volume), G = 25)
nile.data$year[nile.cpts.mosum$cpts]
```

```
[1] 1898
```

Yes: same answer as before.

(f) By investigating the output returned by the `mosum()` function, what are the p-value(s) of the detected change points?

Solution:

```
nile.cpts.mosum$cpts.info
```

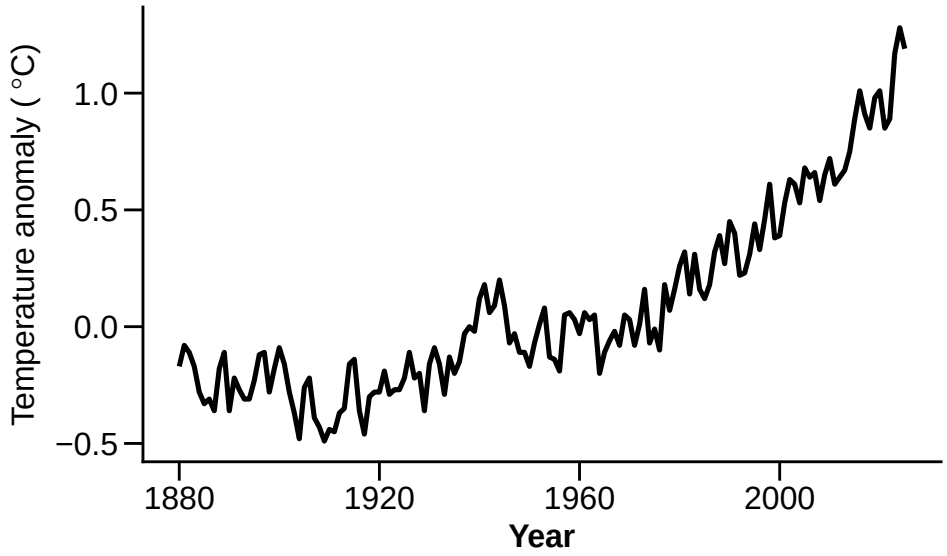
```
  cpts G.left G.right    p.value    jump
1   28     25       25 0.00054662 1.86403
```

The p-value associated to the change is 0.000546620028168743 (this is calculated using the asymptotic null distribution of the test statistic).

Python users: you can use the `ruptures` function `Pelt()` and the `mosum` function `mosum()` for this question. See https://centre-borelli.github.io/ruptures-docs/ and https://pypi.org/project/mosum/ for help.

Question 4: Global yearly mean surface temperature anomalies

Global mean surface temperature series help enable the monitoring of global warming. The warming can be quantified by, for example, a change from a base period used as reference. NASA's GISTEMP land-ocean temperature index provides the annual global mean surface temperature anomalies from 1880 to the present, where anomalies are calculated relative to the 1951–1980 period. The data are shown below, which can be seen to exhibit a gradual upward trend.

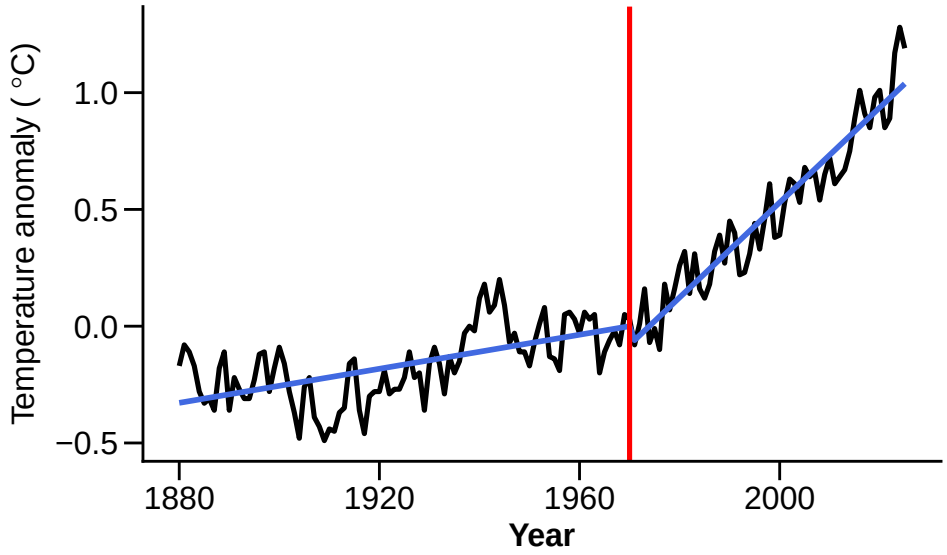


(a) Load the data into `R` from the `temperature_anomalies.csv` file, and plot it. Using e.g. the `lm()` function, fit two linear trends to the data: one from 1880 to 1970, and one from 1971 to the present, and add these lines to the plot, as well as a vertical line through year 1970.

Solution:

```
k <- which(temp.data$year == 1970)

data1 <- temp.data[1:k, ]
data2 <- temp.data[(k + 1):nrow(temp.data), ]
fitted.data <- c(lm(anomaly ~ year, data = data1)$fitted.values,
                 lm(anomaly ~ year, data = data2)$fitted.values)

est.data <- data.frame(year = temp.data$year, anomaly = fitted.data)

trend.plot <- ggplot(data = temp.data, aes(x=year,y=anomaly))+
```

```
  geom_line(data=temp.data,color="black",linewidth=0.9)+
  geom_line(data=est.data,color="royalblue",linewidth=1)+
  theme_classic()+
  labs(x="Year",y=expression('Temperature anomaly ('*~degree*C*')'))+
  theme(axis.text=element_text(size=12),axis.ticks.length=unit(.25, "cm"),
        axis.title=element_text(size=12),title =
          element_text(size=18,face="bold")) +
  geom_vline(xintercept = 1970, color = "red", linewidth =0.9)

trend.plot
```



(b) Using your own functions, and functions from the `changepoint` and `mosum` packages, fit 4 models to the data set;

1. a constant mean model with no change points,
2. a mean change point model,
3. a linear trend model with no change points.
4. a linear trend change point model.

Looking at the data, are there any other models that might be appropriate?

Hint: use the `cpt.reg()` function in `changepoint` for linear trend changes. If you are struggling, the `EnvCpt` R package has everything you need.

Python users: you can use the `Pelt` function and change the `model` parameter.

Solution:

```r
#Fitting the 4 different models:
n <- nrow(temp.data)
m1 <- mean(temp.data$anomaly)
m2 <- cpt.mean((temp.data$anomaly - mean(temp.data$anomaly))/
                 sd(temp.data$anomaly))
m3 <- lm(temp.data$anomaly ~ I(1:n))

T.d <- cbind(temp.data$anomaly, 1, 1:n)
m4 <- cpt.reg(T.d)
```

It might be the case that the data has autocorrelation (nearby observations are correlated). If this is the case, we could consider models that account for this, such as an autoregressive (AR) model with change points in the mean/trend.

We can also do all model fits at once in `EnvCpt` package (note however the mean change now includes a possible change in variance too):

```r
library(EnvCpt)

temp.cpt.models <- envcpt(temp.data$anomaly, models = c("mean", "meancpt",
                                    "trend", "trendcpt"), verbose = FALSE)
```

(c) Pick the "best" change point model from your candidates computed in part (b), and justify your choice. Add the estimated mean/trend function onto a plot of the data.

Hint: the Akaike information criterion (AIC) is given by $\text{AIC} = 2k - 2\log(\hat{L})$, where k is the number of parameters of the model, and $\hat{L}$ is the maximised value of the likelihood function for the model. The smaller the AIC, the "better" the model. For normally distributed data, you can use `dnorm()` in R to compute the likelihood. Compute both the AIC and the BIC (BIC replaces the $2k$ penalty with $k \log n$). Do they select the same model? Which criterion tends to prefer models with more change points, and why?

Solution:

```r
m1.lik <- sum(dnorm(temp.data$anomaly, mean = m1, sd = sd(temp.data$anomaly),
                    log = TRUE))
m1.aic <- 2*2 - 2*m1.lik   # k = 2: mean + variance

mean.m2 <- calc.mean.func(temp.data$anomaly, m2@cpts[1])
m2.lik <- sum(dnorm(temp.data$anomaly, mean = mean.m2,
                    sd = sd(temp.data$anomaly - mean.m2), log = TRUE))
m2.aic <- 2*4 - 2*m2.lik   # k = 4: 2 segment means + 1 change location + variance
```

```
mean.m3 <- m3$fitted.values
m3.lik <- sum(dnorm(temp.data$anomaly, mean = mean.m3,
                    sd = sd(temp.data$anomaly - mean.m3), log = TRUE))
m3.aic <- 2*3 - 2*m3.lik   # k = 3: intercept + slope + variance

mean.m4 <- numeric(0)
cpts <- c(0, m4@cpts)
betas <- param.est(m4)$beta
for(i in 1:nseg(m4)){
  mean.m4 <- c(mean.m4, betas[i,]%*%t(data.set(m4)[(cpts[i]+1):cpts[i+1],-1]))
}

m4.lik <- sum(dnorm(temp.data$anomaly, mean = mean.m4,
                    sd = sd(temp.data$anomaly - mean.m4), log = TRUE))
m4.aic <- 2 * (3 * nseg(m4)) - 2 * m4.lik   # k = 3*nseg: coefs + change locations + variance
aic.values <- c(m1.aic, m2.aic, m3.aic, m4.aic)
aic.values
```

```
[1]  149.96725  -32.93232  -58.92589 -232.69213
```

You can also just directly use `AIC()` and `BIC()` functions in R to compare models fitted using the `EnvCpt` package (note the slight difference in values compared to above, as simultaneous variance changes are also allowed in `EnvCpt`):

```
AIC(temp.cpt.models)
```

```
      mean     meancpt     meanar1     meanar2  meanar1cpt  meanar2cpt
 149.96381  -217.40736          NA          NA          NA          NA
     trend    trendcpt    trendar1    trendar2 trendar1cpt trendar2cpt
 -58.92933  -229.65035          NA          NA          NA          NA
```

```
which.min(AIC(temp.cpt.models))
```

```
trendcpt
       8
```

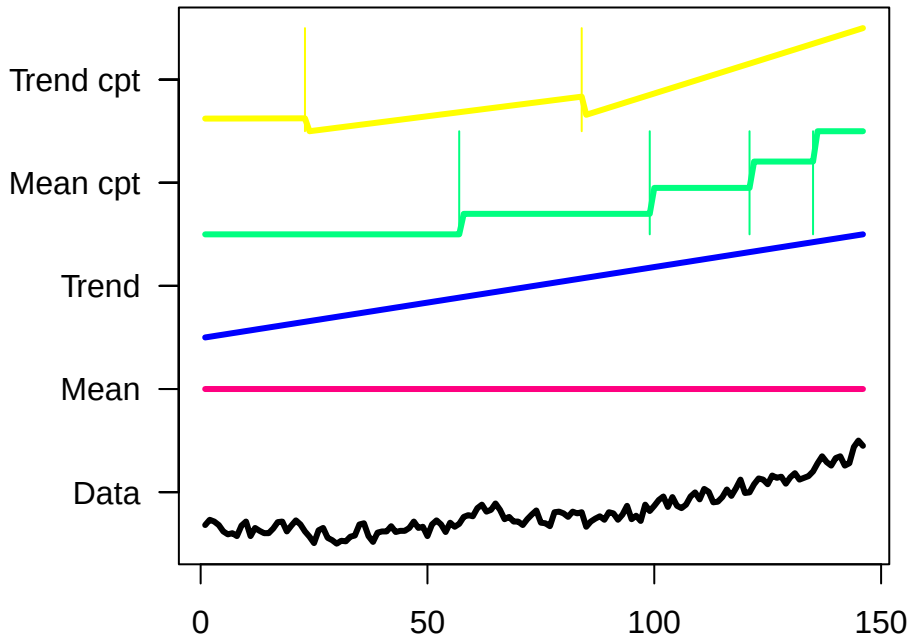
```
BIC(temp.cpt.models)
```

```
      mean     meancpt     meanar1     meanar2  meanar1cpt  meanar2cpt
 155.93103  -175.63687          NA          NA          NA          NA
     trend    trendcpt    trendar1    trendar2 trendar1cpt trendar2cpt
 -49.97851  -196.83067          NA          NA          NA          NA
```

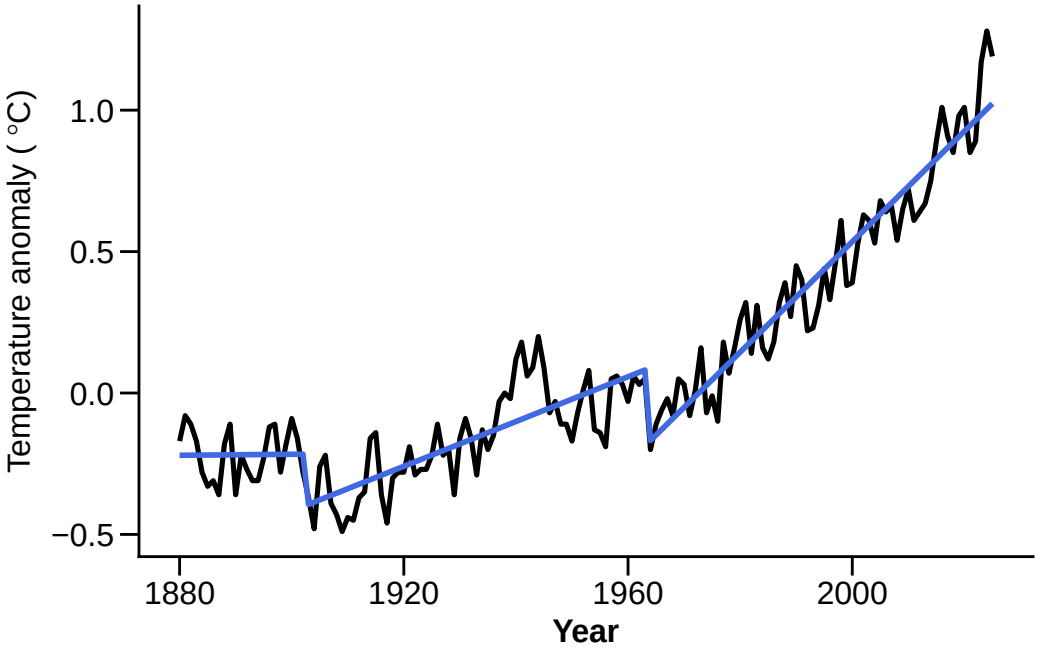
```
which.min(BIC(temp.cpt.models))
```

```
trendcpt
       8
```

```
plot(temp.cpt.models, type = 'fit')
```




```
est.data <- data.frame(year = temp.data$year, anomaly = mean.m4)
trend.plot <- ggplot(data = temp.data, aes(x=year,y=anomaly))+
  geom_line(data = temp.data, color = "black", linewidth = 0.9) +
  geom_line(data = est.data, color = "royalblue", linewidth = 1) +
  theme_classic() +
  labs(x = "Year",y = expression('Temperature anomaly ('*~degree*C*')')) +
  theme(axis.text = element_text(size=12), axis.ticks.length=unit(.25, "cm"),
        axis.title = element_text(size=12), title =
          element_text(size=18,face="bold"))
trend.plot
```

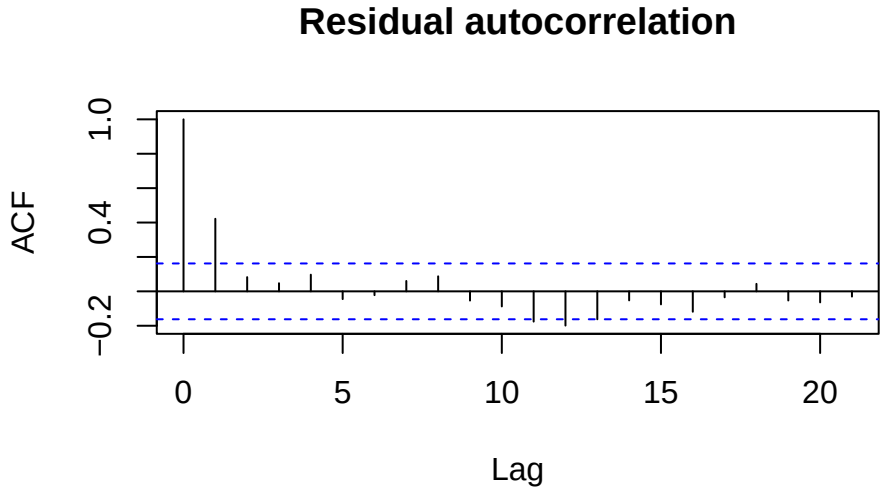


Here the AIC and BIC need not agree. BIC penalises each extra parameter more heavily than AIC (its penalty $k \log n$ exceeds $2k$ whenever $n > e^2 \approx 7.4$), so BIC tends to prefer **simpler** models with **fewer** change points, while AIC is more willing to add them. Neither is universally correct: roughly speaking, AIC aims for the best predictive fit, while BIC aims to recover the true model when it is among the candidates.

(d) How good is the fit of your chosen model? Compute the residuals (the data minus the fitted mean/trend), and plot their autocorrelation function using `acf()`. Do the residuals look like independent noise, or is there leftover structure? Why might autocorrelated noise be a problem for change point detection – what would you expect to happen to the number of detected changes if you (wrongly) assume the noise is independent?

Solution: Using the fitted values from the trend-change model (`mean.m4`) computed above:

```
resid.m4 <- temp.data$anomaly - mean.m4
acf(resid.m4, main = "Residual autocorrelation")
```



The residuals show clear positive autocorrelation at short lags – neighbouring years are correlated, so the residuals are **not** independent white noise. This matters because CUSUM, MOSUM and PELT all assume independent noise: positive autocorrelation lets the series drift in a way that locally mimics small level shifts, inflating the test statistic under the null. The practical consequence is that we tend to detect **too many** change points (an inflated false-alarm rate). Allowing for the autocorrelation – for example with an autoregressive-plus-change-point model, as in `EnvCpt` – protects against this.

Question 5 (Bonus): Multivariate data

(a) Suppose we want to find mean change points in a time series $\{X_t\}_{t=1}^n$, where $X_t = (X_{1t}, \dots, X_{pt})^\mathsf{T}$ is a p-dimensional vector. One approach is to calculate a test statistic for each variable, and then combine the results across variables to give a single test statistic. For example, for the i-th variable of $\{X_t\}_{t=1}^n$, denoted $\{X_{it}\}_{t=1}^n$, the MOSUM test statistic is given by

$$T_G(k,i) = \frac{1}{\sqrt{2G}} \left(\sum_{t=k+1}^{k+G} X_{it} - \sum_{t=k-G+1}^{k} X_{it} \right).$$

Then, the MOSUM statistic $ T_{G}(k)$ for a change point in $\{X_t\}_{t=1}^n$ can be computed by aggregating the $\{T_G(k,i)\}_{i=1}^p$ using some aggregating function f, so that

$$T_G(k) = f\left(T_G(k,1), \dots, T_G(k,p)\right).$$

What possibilities could we use for the aggregating function f?

Solution:

We could use any appropriate norm, such as the L_1 or L_2 norm:

- $f(x_1, \ldots, x_p) = (\sum_{i=1}^{p} x_i^2)^{1/2}$ (the L_2 norm)
- $f(x_1, \ldots, x_p) = \max_{i=1,\ldots,p} |x_i|$ (the L_∞ norm)

(b) Using your aggregating function f, write a function to compute the MOSUM or CUSUM statistic for a change in the mean vector of a multivariate time series.

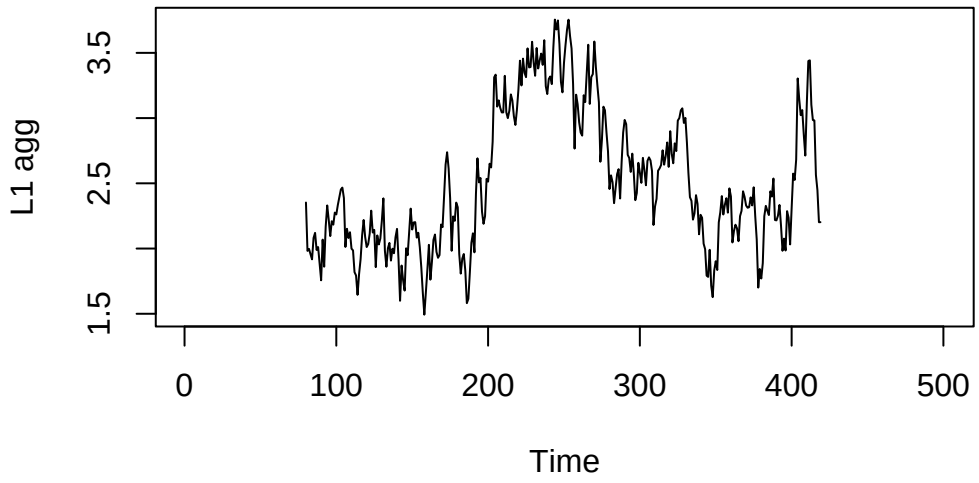
Solution:

```
multi.MOSUM <- function(x, G, agg = c("L1", "L2")[1]){

  matrix.MOSUM <- abs(apply(x, 2, MOSUM.calc2, G = G))

  if(agg == "L2"){
    x.MOSUM <- sqrt(rowSums(matrix.MOSUM^2))
  }else{
    x.MOSUM <- apply(matrix.MOSUM, 1, max)
  }

  return(x.MOSUM)

}
```

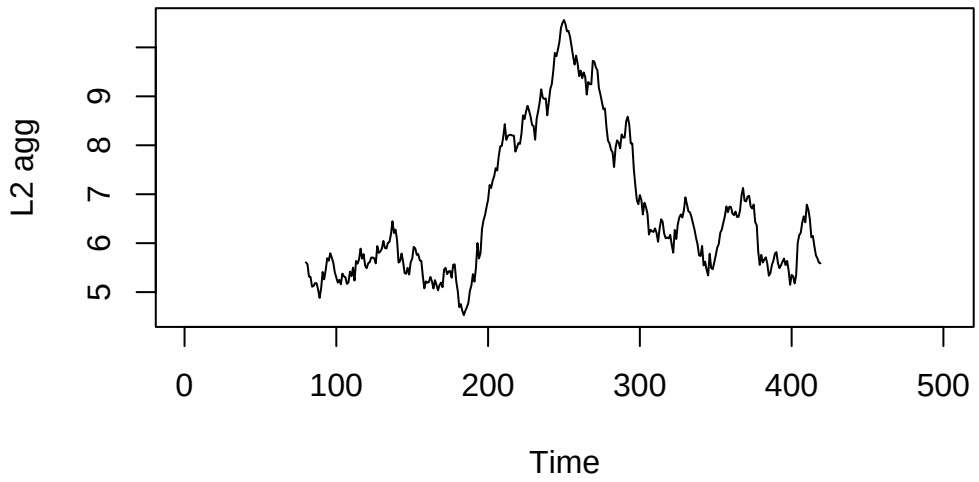
(c) Simulate a data set of length $n = 500$ and dimension $p = 40$ as follows. Generate $\{X_t\}_{t=1}^{250}$ from the standard p-dimensional normal distribution, and generate $\{X_t\}_{t=251}^{500}$ from the p-dimensional normal distribution with identity covariance matrix, and mean vector given by $\mu = 1.4 \times (1/\sqrt{p}, \ldots, 1/\sqrt{p})^\mathsf{T}$. Use the method from part (b), with $G = 80$, to compute the test statistic for a change in mean. Is the change point easy to see?

Solution:

```
set.seed(1)
x <- matrix(rnorm(40*500), nrow = 500, ncol = 40)

x[251:500,] <- x[251:500,] + 1.4/sqrt(40)

plot.ts(multi.MOSUM(x, G = 80, agg = "L1"), ylab = "L1 agg")
```




```
plot.ts(multi.MOSUM(x, G = 80, agg = "L2"), ylab = "L2 agg")
```

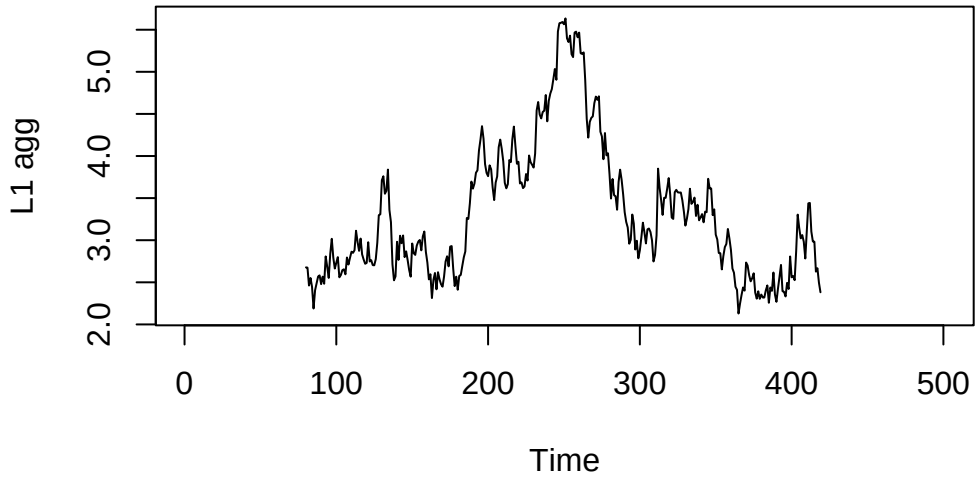


(d) Re-do part (c), but generate $\{X_t\}_{t=251}^{500}$ from the $p = 200$-dimensional normal distribution with identity covariance matrix, and mean vector given μ with first 2 entries equal to 0.8, and last $p-2$ entries equal to 0. Is the change point easy to see? Compare with part (c): which aggregation was more effective in each case, and can you explain why, in terms of whether the change is *dense* (spread across many coordinates) or *sparse* (confined to a few)?

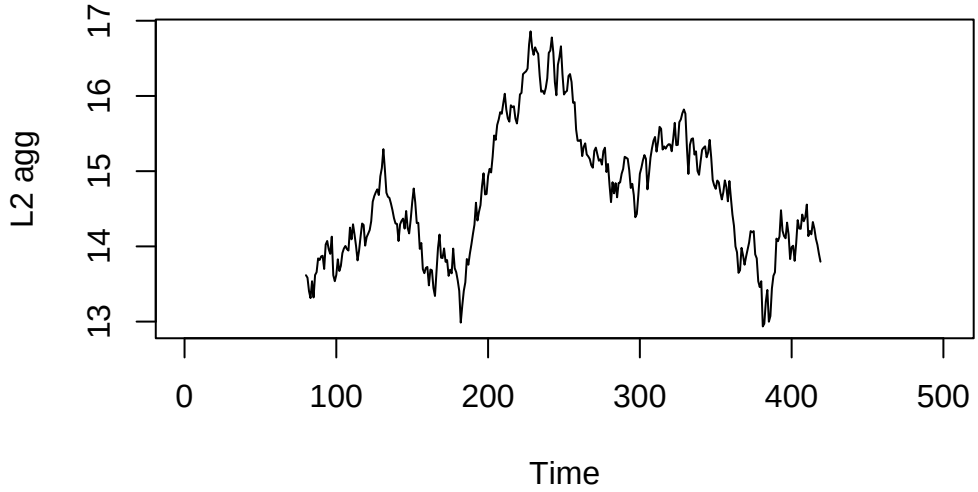
Solution:

```
set.seed(1)
x <- matrix(rnorm(200*500), nrow = 500, ncol = 200)
```

```r
x[251:500,1:2] <- x[251:500,1:2] + 0.8

plot.ts(multi.MOSUM(x, G = 80, agg = "L1"), ylab = "L1 agg")
```




```r
plot.ts(multi.MOSUM(x, G = 80, agg = "L2"), ylab = "L2 agg")
```



In part (c) the change is **dense** – all 40 coordinates shift by a small amount. The L_2 aggregation (`agg = "L2"`) adds up these many small signals, so the change stands out clearly. In part (d) the change is **sparse** – only 2 of the 200 coordinates shift. Now the L_2 aggregation drowns those two signals in the noise of the other 198 coordinates, whereas the maximum aggregation (the `agg = "L1"` option, which takes the coordinate-wise maximum) isolates the few large components and detects the change much more clearly. The lesson is to match the aggregation

to how the change is spread across dimensions: L_2-type aggregation for dense changes, and max / L_∞-type aggregation for sparse ones.